

5.2 Tipovi podataka u C++

5.2.1 Deklaracije podataka u C++

5.2.2 Statički skalarni tipovi podataka

5.2.3 Dinamički tipovi podataka

5.2 Tipovi podataka u C++

5.2.1 Deklaracije podataka u C++

Svaki program koji nešto radi, sadrži podatke koje obrađuje. Ti podaci mogu biti u obliku varijabli i konstanti. Varijable mogu mijenjati svoju vrijednost, dok su konstante postojane u toku izvođenja programa.

5.2.1.1 Deklaracija varijable

Svaku varijablu koju koristimo u programu moramo deklarirati. Deklaracija varijable može se nalaziti ispred funkcije *main* - u tom slučaju se radi o tzv. globalnoj varijabli, odnosno, varijabli dostupnoj u svim dijelovima programa. Također varijablu možemo deklarirati i unutar bilo koje funkcije u programu, štoviše, varijable možemo deklarirati u svim dijelovima programa (to je novina u C++ u odnosu na C).

Varijablu deklariramo tako da navedemo najprije njen tip, a zatim njen naziv :

`<tip> <naziv>;`

odnosno (zajedno s inicijalizacijom) :

`<tip> <naziv> = <početna vrijednost>;`

Primjer :

```
#include <iostream.h>
int globalna;                // globalna varijabla tipa int (cjelobrojna)
void main () {
    char lokalna='A';        // lokalna varijabla tipa char (znakovna)
    globalna = 3;
    for (int i=1;i<=10;i++){  // varijablu možemo deklarirati i ovdje
        int b=7;             // pa čak i ovdje
        cout << i << endl;
    }
}
```

5.2.1.2 Deklaracija konstante

Konstante su podaci kojima (barem načelno) ne možemo promijeniti vrijednost u toku izvođenja programa. C++ za deklaraciju konstanti ključnu riječ *const*, za razliku od C-a, koji koristi pretprocesorsku naredbu *#define*. U odnosu na deklaraciju varijable, razlika je u tome

što ispred tipa konstante navodimo ključnu riječ *const*, i u tome što konstantu obavezno moramo inicijalizirati (pridružiti joj vrijednost) :

```
const <tip> <naziv> = <početna vrijednost>;
```

Primjer :

```
const double e = 2.71;
```

Konstante deklarirane sa *const* predstavljaju tzv. imenovane konstante, i njima pristupamo preko njihovog naziva, kao i varijablama. Na primjer :

```
cout << e << endl;    // ispisuje se 2.71
```

Osim imenovanih konstanti, u programu možemo koristiti i konstante koje navodimo po vrijednosti, odnosno, svaki podatak koji u programu navodimo po vrijednosti, predstavlja konstantu :

```
cout << "Ispis na ekranu" << endl;
```

Ovdje tekst u navodnicima ("Ispis na ekranu") predstavlja konstantu koju navodimo po vrijednosti. Navodnici sugeriraju da se radi o znakovnom nizu koji zadajemo kao konstantu.

U slučaju da zadajemo numeričku vrijednost u oktalnom, odnosno heksadekadskom brojevnom sustavu, koristimo odgovarajuće prefikse : 0 (nula), odnosno 0x :

```
int okt=010;    // dek. 8  
int hex=0x10;  // dek. 16
```

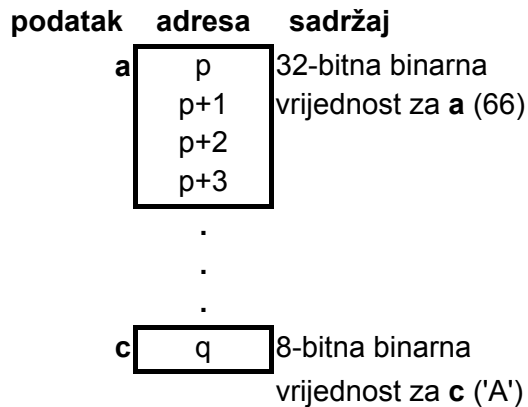
5.2.1.3 Tip podatka

Radnu memoriju računala možemo promatrati kao dugačak niz jednostavnih podataka, bajtova, od kojih svaki ima svoju adresu. Deklaracijom podatka (varijable ili konstante) rezerviramo određeni prostor u radnoj memoriji računala, gdje će biti u binarnom obliku smještena vrijednost tog podatka. Veličina tog memorijskog prostora ovisi o tipu podatka, dok nas stvarna fizička adresa u većini slučajeva ne zanima, odnosno, prepuštamo sustavu za alokaciju memorije programa prevoditelja da je odredi. Prema tome, deklarirane varijable i konstante možemo promatrati kao imenovane memorijske prostore.

Primjer :

```
int a=66;        // rezervira se prostor veličine 4 bajta za podatak tipa int,  
char c='A';      // a jedan bajt za podatak tipa char
```

U radnoj memoriji računala rezerviran je odgovarajući memorijski prostor :



Slika 7.1 : varijable kao imenovani memorijski prostori

Prilikom ispisa podatka na standardnoj izlaznoj jedinici (odnosno, kod tzv. *konzolnog izlaza*) dolazi do odgovarajuće konverzije binarnog sadržaja memorijskog prostora varijable u odgovarajući niz znakova. Na primjer :

```
cout << a << endl;      // ispisuje se brojka, 66
cout << c << endl;      // ispisuje se znak, A
```

Tip konverzije podatka određen je tipom podatka, a načelno govorimo o 5 osnovnih tipova konverzije (A, X – znakovne, I – cjelobrojna, F – konverzija brojeva u pokretnom zarezu i E – konverzija brojeva u eksponencijalnoj formi; vidi odgovarajuća o ulazu i izlazu podatka). Prilikom unosa podatka dolazi do obratne konverzije; niz znakova sa standardne ulazne jedinice (tzv. konzolni ulaz) pretvara se u odgovarajući binarni sadržaj, prema tipu podatka :

```
int e; char f;
cin >> e;      // konverzija tipa I
cin >> f;      // konverzija tipa A
```

U slijedećem primjeru koristimo tzv. operator dodjele tipa :

```
cout << (char) a << endl;    // ispisuje se znak, B, ASCII kod 66
cout << (int) c << endl;     // ispisuje se broj, 65, ASCII kod znaka A
```

Pomoću operatora dodjele tipa određujemo da će se nekim memorijskim prostorom rukovati kao da se radi o podatku tipa koji (privremeno) dodjeljujemo, umjesto tipa koji je podatku dodijeljen prilikom deklaracije.

Općenito možemo reći da tipovi podatka određuju veličinu memorijskog prostora i način rukovanja podacima.

5.2.2 Statički skalarni tipovi podataka

5.2.2.1 Linearno uređeni

5.2.2.1.1 Cjelobrojni (INTEGER)

Tip podatka	Veličina u bitovima	Opseg vrijednosti
short int	16	-32768..32767
unsigned int	32	0..4294967295
int	32	-2147483648..2147483647
long	32	-2147483648..2147483647
unsigned long	32	0..4294967295

5.2.2.1.2 Znakovni (CHARACTER)

Tip podatka	Veličina u bitovima	Opseg vrijednosti
char	8	-128..127
signed char	8	-128..127
unsigned char	8	0..255

5.2.2.1.3 Nabrojivi (ENUMERATED)

Nabrojivi tip podataka omogućuje nam da sami definiramo moguće vrijednosti podataka. Za deklaraciju nabrojivog tipa podataka (pobrojenja) koristimo ključnu riječ *enum* :

```
enum <tip> {v1,v2,...,vn}
```

Primjer:

```
enum tdan {pon,uto,sri,cet,pet,sub,ned};
```

deklarira tip podataka *tdan*. Varijable tipa *tdan* mogu sadržavati jednu od navedenih vrijednosti za dane u tjednu :

```
tdan dan;                // dan je varijabla tipa tdan
dan = pet;               // pridružujemo vrijednost varijabli dan
(*cout << dan << endl;*) // ne postoji odgovarajuća konverzija za podatke
                        // nabrojivog tipa, pa ispis nije moguć
cout << (int) dan << endl; // 4; numerička vrijednost varijable dan
```

prema tome, vrijednosti pobrojenja zapravo predstavljaju nazive za odgovarajuće numeričke vrijednosti. Prvi element pobrojenja ima numeričku vrijednost 0 :

```
cout << (int) pon << endl; // 0
cout << (int) ned << endl; // 6
```

osim, ako se vrijednosti ne zadaju, kao u slijedećem primjeru :

```
enum tstanje {lose=0,slabo=5,srednje=10,dobro=15,odlicno=20};
tstanje stanje;
stanje = dobro;
cout << int(stanje) << endl; // 15
```

5.2.2.1.4 Logički (LOGICAL)

Za razliku od Pascala, u C++ logički izrazi daju cjelobrojni rezultat, i to rezultat 0 koji predstavlja logičku laž, odnosno rezultat 1 koji predstavlja logičku istinu. Također, sve vrijednosti različite od nule tretiraju se kao logička istina.

Primjer :

```
cout << (2==2) << endl;           // ispisuje se 1
if (-5){
    cout << "Ovaj uvjet je zadovoljen." << endl;
}
```

Logički tip podataka, *bool*, predstavlja pobrojenje s elementima *false* i *true*, nije podržan od svih C++ prevoditelja, a definira se ovako :

```
enum bool {false, true}
```

5.2.2.2 Linearno neuređeni tipovi podataka

5.2.2.2.1 Realni tipovi (REAL)

Tip podatka	Veličina u bitovima	Opseg vrijednosti
float	32	3.4E-38.. 3.4E38
double	64	1.7E-308..1.7E308
long double	80	3.4E-4932..1.1E4932

5.2.2.3 Statički strukturirani tipovi podataka

5.2.2.3.1 Znakovni niz (CHARACTER STRING)

char [<veličina niza>]

Znakovni niz definira se kao jednodimenzionalno polje znakova :

Primjer :

```
char a[12];           // najviše 11 znakova + znak za terminiranje niza
strcpy(a, "Niz znakova");
cout << a[0] << endl; // 'N'; prvi znak niza
```

C++ radi s tzv. nul-terminiranim znakovnim nizovima, odnosno, kraj niza znakova označen je znakom \0 (znak s ASCII kodom 0), pa je kod deklaracije potrebno o tome voditi računa.

Niz (ARRAY)

<tip> [v1] ... [vn]

Primjer 1 (polje za Križić-kružić) :

```
char polje [3][3];
polje [1][1] = "X" // Srednje polje, jer indeksi počinju s 0!
```

Mogući su višedimenzionalni nizovi, s tim da se svaka od dimenzija zadaje u vlastitom paru uglatih zagrada.

5.2.2.3.2 Slogovi (zapisi; RECORD)

Slogovi uključuju varijable različitih naziva i tipova, koji se odnose na određenu jedinicu (entitet) promatranja.

Primjer :

<i>Paganini</i>	<i>Niccolo</i>	<i>1742</i>	<i>1782</i>	<i>violina</i>
<i>Liszt</i>	<i>Franz Ferenc</i>	<i>1811</i>	<i>1886</i>	<i>klavir</i>
<i>Boccherini</i>	<i>Luigi</i>	<i>1743</i>	<i>1805</i>	<i>violončelo</i>
<i>Bach</i>	<i>Johann Sebastian</i>	<i>1685</i>	<i>1750</i>	<i>orgulje</i>

Definicije niza polja unose se pomoću ključne riječi *struct* :

```
struct t_glazbenik {
    char prezime[20];
    char ime [20];
    int godina_r, godina_s;
    char instrument [16];
};
t_glazbenik glazbenik;
glazbenik.godina_r =1742;
```



```
strcpy (glazbenik.prezime, "Paganini");
```

5.2.3 Dinamički tipovi podataka

5.2.3.1 Dinamički tipovi podataka s promjenjivom veličinom

5.2.3.1.1 Dinamički podaci tipa datoteke

Koriste se klase *fstream*, *ifstream* i *ofstream* :

<naziv_klase>.<naziv_datotečnog objekta>

Primjer:

```
fstream dat;
```

C++ tretira datoteke kao objekte klase *fstream*, odnosno iz nje izvedenih klasa *ifstream* (definira ulazne datoteke) i *ofstream* (izlazne datoteke).

5.2.3.1.2 Dinamički podaci tipa pokazivača (promjenjiva struktura)

Pokazivač je varijabla koja sadrži adresu neke druge varijable ili funkcije. Tada tu varijablu/funkciju nazivamo dinamičkom. Deklaracija pokazivača određuje tip na koji pokazuje i ima slijedeći format :

tip_na_koji_pokazuje *ime_pokazivaca;

Primjer:

```
int *int_pok;      /* Pokazivač na cijeli broj */
double *dbl_pok;   /* Pokazivač na realni broj */

int_pok = new int; /* Rezervira se memorijski prostor za novu dinamičku varijablu
                  čiju adresu dobiva pokazivač*/
delete int_pok;    /* Oslobađa se memorijski prostor zauzet pomoću new */
```